

Lecture: Counting Sort Algorithm — From Zero to Advanced

1. Introduction

Counting Sort is a **non-comparison-based sorting algorithm** that works by **counting the frequency of elements**.

Instead of comparing elements like Bubble or Quick Sort, it uses an auxiliary array to count how many times each element appears and then reconstructs the sorted array.

Definition:

Counting Sort counts the occurrences of each element and uses this count to place elements directly into their correct sorted position.

Real-life analogy:

Imagine sorting exam scores (0–100). Instead of comparing students' scores one by one, you prepare 101 boxes (one for each possible score), count how many students scored each mark, and then list them in order.

2. Key Characteristics

- **Type:** Non-comparison-based Sorting
- **Input Requirement:** Elements must be **integers within a known range** (0 to k)
- **Direction:** Counts occurrences → cumulative counts → place elements
- **Time Complexity:**
 - Best: $O(n + k)$
 - Average: $O(n + k)$
 - Worst: $O(n + k)$
- **Space Complexity:** $O(n + k)$ (for count and output arrays)
- **Stability:** Yes (if implemented carefully)

3. Step-by-Step Working

1. Find the maximum value in the array (say k).
2. Create a count array of size k+1 initialized to 0.
3. Count the occurrences of each element in the input array.

4. Modify the count array to store cumulative counts.
5. Place each element in its correct position in the output array using cumulative counts.
6. Copy the output back to the original array (sorted).

4. Pseudocode

CountingSort(arr):

n = length(arr)

max_val = maximum element in arr

count = array of size (max_val+1) initialized to 0

output = array of size n

for i = 0 to n-1:

 count[arr[i]] += 1

for i = 1 to max_val:

 count[i] += count[i-1]

for i = n-1 down to 0:

 output[count[arr[i]] - 1] = arr[i]

 count[arr[i]] -= 1

return output

5. Complexity Analysis

- **Best Case:** $O(n + k)$
- **Worst Case:** $O(n + k)$
- **Average Case:** $O(n + k)$

- **Space Complexity:** $O(n + k)$
- **Stability:** Yes

Works faster than $O(n \log n)$ algorithms when k (range of numbers) is not much larger than n .

6. Handling Edge Cases

1. **Empty list** → already sorted.
2. **Single element list** → already sorted.
3. **Negative numbers** → standard Counting Sort doesn't work (needs modification to shift values).
4. **Large range of values** → inefficient due to memory use.

7. Variations of Counting Sort

- **Stable Counting Sort** → ensures equal elements keep their order.
- **Modified Counting Sort for negative numbers** → by shifting indices.
- **Radix Sort** → uses Counting Sort as a subroutine for digit-wise sorting.

8. When to Use Counting Sort

Best suited for:

- Sorting integers in a small or moderately sized range.
- Situations where stability matters (like sorting IDs, grades, etc.).

Avoid if:

- Data contains large numbers (range too big → high memory usage).
- Data contains floating-point numbers or strings (unless mapped to integers).

9. Advanced Usage

- As a subroutine in **Radix Sort** for digit-wise sorting.
- Used when **time complexity below $O(n \log n)$** is needed.
- Useful in **histogram-based problems** and frequency analysis.

10. Real-World Applications

- **Election vote counting** → tally votes efficiently.

- **Counting grades/scores** in schools.
- **Digit sorting in Radix Sort** (phone numbers, postal codes).
- **Frequency analysis in strings** (like anagrams, character counts).

11. Summary Table

Feature	Counting Sort
Works on sorted?	Yes
Works on unsorted?	Yes (if integers in range)
Time Complexity	Best: $O(n + k)$, Worst: $O(n + k)$, Avg: $O(n + k)$
Space Complexity	$O(n + k)$
Stable?	Yes (with correct implementation)

12. Practice Tasks

1. Implement **basic Counting Sort** in Python.
2. Modify Counting Sort to work with **negative numbers**.
3. Compare Counting Sort with Quick Sort and Merge Sort for arrays with small integer ranges.
4. Implement **stable Counting Sort** that preserves order of duplicates.
5. Use Counting Sort as a subroutine in **Radix Sort** for sorting phone numbers.

13. Closing Thought

Counting Sort is a powerful non-comparison algorithm that can achieve **linear time complexity** when the range of input values is not much larger than the number of elements.

Though it's not suitable for all data types, its use in **Radix Sort and frequency-based problems** makes it a vital algorithm to master.